MODULE – 2

Embedded Systems, Operational and non-operational quality attributes, Embedded Systems-Application and Domain specific, Hardware Software Co-Design and Program Modelling (excluding UML), Embedded firmware design and development (excluding C language). (Text 2: Ch-3, Ch-4, Ch-7 (Sections 7.1, 7.2 only), Ch-9 (Sections 9.1, 9.2, 9.3.1, 9.3.2 only)

CHARACTERISTICS OF AN EMBEDDED SYSTEM

Eembedded systems possess certain specific characteristics and these characteristics are unique to each embedded system. Some of the important characteristics of an embedded system are:

1. Application and domain specific
2. Reactive and Real Time
3. Operates in harsh environments
4. Distributed
5. Small size and weight
6. Power concerns

Application and Domain Specific

Each embedded system are developed to do the intended specific functions only. They cannot be used for any other purpose. For example, the embedded control unit of microwave oven cannot be replaced with air conditioners embedded control unit, because the embedded control units of microwave oven and air conditioner are specifically designed to perform certain specific tasks. Also an embedded control unit developed for a particular domain says telecom cannot be replaced with another control unit designed to serve another domain like consumer electronics.

Reactive and Real Time

Embedded systems are in constant interaction with the Real world through sensors and user-defined input devices which are connected to the input port of the system. Any

changes happening in the Real world (which is called an Event) are captured by the sensors or input devices in Real Time and the control algorithm running inside the unit reacts in a designed manner to bring the controlled output variables to the desired level. A Real Time system should not miss any deadlines for tasks or operations. Embedded applications or systems which are mission critical, like flight control systems, Antilock Brake Systems (ABS), etc. are examples of Real Time systems.

Operates in Harsh Environment

It is not necessary that all embedded systems should be deployed in controlled environments. The environment in which the embedded system deployed may be a dusty one or a high temperature zone or an area subject to vibrations and shock. Systems placed in such areas should be capable to withstand all these adverse operating conditions. The design should take care of the operating conditions of the area where the system is going to implement. For example, if the system needs to be deployed in a high temperature zone, then all the components used in the system should be of high temperature grade.

Distributed

The term distributed means that embedded systems may be a part of larger systems. Many numbers of such distributed embedded systems form a single large embedded control unit. An automatic vending machine is a typical example for this. The vending machine contains a card reader, a vending unit, etc. Each of them are independent embedded units but they work together to perform the overall vending function. Another example is the Automatic Teller Machine (ATM). An ATM contains a card reader embedded unit, responsible for reading and validating the user's ATM card, transaction unit for performing transactions, a currency counter for dispatching/vending currency to the authorised person and a printer unit for printing the transaction details.

Small Size and Weight

An embedded system that is compact in size and has light weight will be desirable or more popular than one that is bulky and heavy. Ex. Currently available cell phones. The cell phones that have the maximum features are popular but also their size and weight is an important characteristic

Power Concerns

It is desirable that the power utilization and heat dissipation of any embedded system be low. If more heat is dissipated then additional units like heat sinks or cooling fans need

to be added to the circuit. If more power is required then a battery of higher power or more batteries need to be accommodated in the embedded system.

QUALITY ATTRIBUTES OF EMBEDDED SYSTEM

These are the attributes that together form the deciding factor about the quality of an embedded system. There are two types of quality attributes are:-

Operational Quality Attributes:

These are attributes related to operation or functioning of an embedded system. The way an embedded system operates affects its overall quality.

Non-Operational Quality Attributes:

These are attributes not related to operation or functioning of an embedded system. The way an embedded system operates affects its overall quality. These are the attributes that are associated with the embedded system before it can be put in operation.

OPERATIONAL QUALITY ATTRIBUTES

(i) to the operating person and environment due to the breakdown of an embedded system

Response

- Response is a measure of quickness of the system.
- It gives you an idea about how fast your system is tracking the input variables.
- Most of the embedded system demand fast response which should be real-time.

(ii) Throughput

- Throughput deals with the efficiency of system.
- It can be defined as rate of production or process of a defined process over a stated period of time.
- For example, In the case of a Card Reader, throughput means how many transactions the Reader can perform in a minute or in an hour orin a day. Throughput is generally measured in terms of 'Benchmark'.

(iii) Reliability

- Reliability is a measure of how much percentage you rely upon the proper functioning of the system .

• Mean Time between failures and Mean Time To Repair are terms used in defining system reliability.
• MTBF gives the frequency of failures in hours/weeks/months. MTTR specifies how long the system is allowed to be out of order following a failure. For an embedded system with critical application need, it should be of the order of minutes.

(iv) Maintainability

• Maintainability deals with support and maintenance to the end user or a client in case of technical issues and product failures or on the basis of a routine system checkup
• It can be classified into two types :-
1. Scheduled or Periodic Maintenance: This is the maintenance that is required regularly after a periodic time interval. Example : Periodic Cleaning of Air Conditioners, Refilling of printer cartridges.
2. Maintenance to unexpected failure: This involves the maintenance due to a sudden breakdown in the functioning of the system. Example: If the paper feeding part of the printer fails the printer fails to print and it requires immediate repairs to rectify this problem.

(v) Security

• Confidentiality, Integrity and Availability are three major measures of information security.
• Confidentiality deals with protection data from unauthorized disclosure. Integrity gives protection from unauthorized modification. Availability gives protection from unauthorized user.
• Certain Embedded systems have to make sure they conform to the security measures.

(vi) Safety

Safety deals with the possible damage that can happen to the operating person and environment due to the breakdown of an embedded system or due to the emission of hazardous materials from the embedded products.

Non-Operational Quality Attributes

The important Non-Operational quality attributes are:

1. Testability & Debug-ability

2. Evolvability

3. Portability

4. Time to prototype and market

5. Per unit and total cost.

Testability & Debug-ability:

Testability deals with how easily one can test his/her design, application and by which means he/she can test it. For an embedded product, testability is applicable to both the embedded hardware and firmware. Embedded hardware testing ensures that the peripherals and the total hardware functions in the desired manner, whereas firmware testing ensures that the firmware is functioning in the expected way. Debug-ability is a means of debugging the product as such for figuring out the probable sources that create unexpected behaviour in the total system. Debug-ability has two aspects in the embedded system development context, namely, hardware level debugging and firmware level debugging. Hardware debugging is used for figuring out the issues created by hardware problems whereas firmware debugging is employed to figure out the probable errors that appear as a result of flaws in the firmware

Evolvability:

For an embedded system, .the quality attribute 'Evolvability' refers to the ease with which the embedded product (including firmware and hardware) can be modified to take advantage of new firmware or hardware technologies.

Portability:

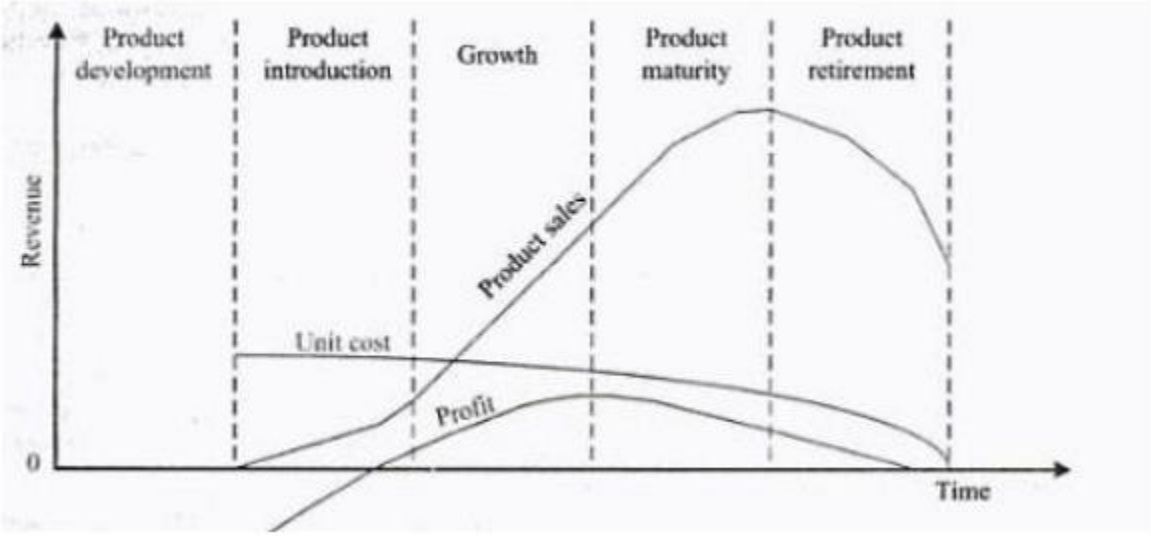
Portability is a measure of'system independence'. An embedded product is said to be portable if the product is capable of functioning 'as such' in various environments, target processors/controllers and embedded operating systems. A standard embedded product should always be flexible and portable.

Time-to-Prototype and Market:

Time-to-market is the time elapsed between the conceptualisation of a product and the time at which the product is ready for selling (for commercial product) or use (for non-commercial products). The commercial embedded product market is highly competitive and time to market the product is a critical factor in the success of a commercial embedded product. The time to prototype is also another critical factor that refers to the time required to develop the working model of a system. In order to shorten the time to prototype, make use of all possible options like the use of off-the-shelf components, re-usable assets, etc.

Per Unit Cost and Revenue:

Cost is an important factor which needs to be carefully monitored. Proper market study and cost benefit analysis should be carried out before taking decision on the per unit cost of the embedded product. When the product is introduced in the market, for the initial period the sales and revenue will be low. There won't be much competition when the product sales and revenue increase. During the maturing phase, the growth will be steady and revenue reaches highest point and at retirement time there will be a drop in sales volume.



Embedded Systems – Application and Domain specific

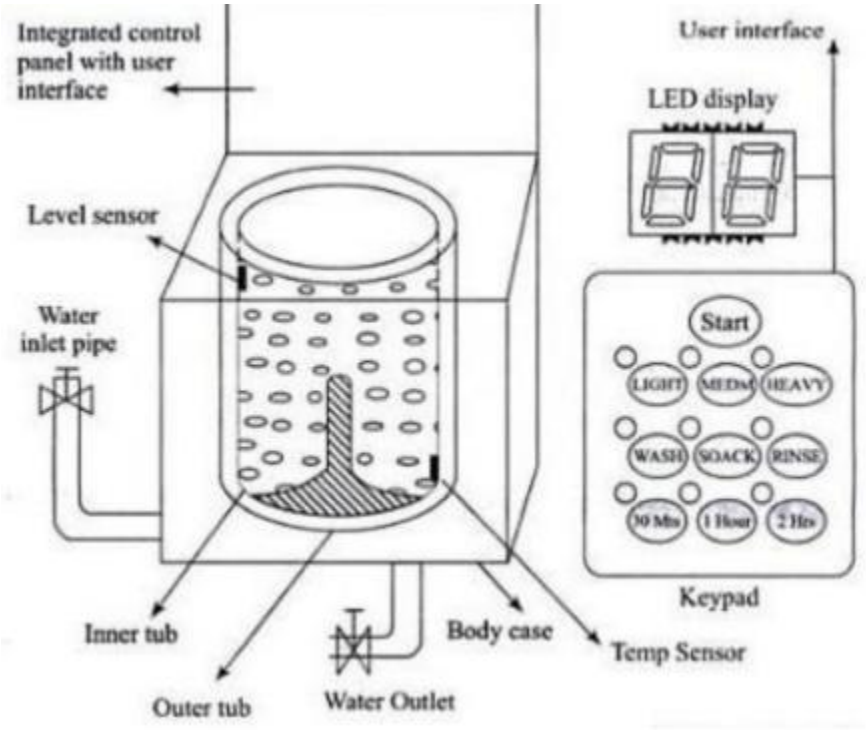
WASHING MACHINE—APPLICATION-SPECIFIC EMBEDDED SYSTEM:

Washing machine is a typical example of an embedded system providing extensive support in home automation applications. An embedded system contains sensors, actuators, control unit and application-specific user interfaces like keyboards, display units, etc. Washing machine contains all these components, among them some are visible and some of them are invisible.

The actuator part of the washing machine consists of a motorised agitator, tumble tub, water drawing pump and inlet valve to control the flow of water into the unit.

The sensor part consists of the water temperature sensor, level sensor, etc. The control part contains a microprocessor/controller based board with interfaces to the sensors

and actuators. The sensor data is fed back to the control unit and the control unit generates the necessary actuator outputs. The control unit also provides connectivity to user interfaces like keypad for setting the washing time, selecting the type of material to be washed like light, medium, heavy duty, etc. User feedback is reflected through the display unit and LEDs connected to the control board. The functional block diagram of a washing machine is shown in Fig.



Washing machine comes in two models, namely, top loading and front loading machines. In top loading models the agitator of the machine twists back and forth and pulls the cloth down to the bottom of the tub, then push them back up to the top of the tub where the agitator grabs them again and repeats the mechanism.

In the front loading machines, the clothes are tumbled and plunged into the water over and over again. This is the first phase of washing.

In the second phase of washing, water is pumped out from the tub and the inner tub uses centrifugal force to wring out more water from the clothes by spinning at several hundred Rotations Per Minute (RPM). This is called a ‘Spin Phase’.

During the spin cycle the inner tub spins, and forces the water out through a number of holes to the stationary outer tub from which it is drained off through the outlet pipe.

The basic controls of a washing machine consist of a timer, cycle selector mechanism, water temperature selector, load size selector and start button.

The mechanism includes the motor, transmission, clutch, pump, agitator, inner tub, outer tub and water inlet valve. Water inlet valve connects to the water supply line using at home and regulates the flow of water into the tub.

The integrated control panel consists of a microprocessor/controller based board with I/O interfaces and a control algorithm running in it.

Input interface includes the keyboard which consists of wash type selector to configure different washing stages namely, Wash, Spin and Rinse, cloth type selector namely Light, Medium, Heavy duty and washing time setting, etc.

The output interface consists of LED/LCD displays, status indication LEDs, etc. connected to the I/O bus of the controller.

The other types of I/O interfaces which are invisible to the end user are different kinds of sensor interfaces, namely, water temperature sensor, water level sensor, etc. and actuator interface including motor control for agitator and tub movement control, inlet water flow control, etc.

Automotive Embedded System (AES)

• The Automotive industry is one of the major application domains of embedded systems.

• Automotive embedded systems are the one where electronics take control over the mechanical system. Ex. Simple viper control.

• The number of embedded controllers in a normal vehicle varies somewhere between 20 to 40 and can easily be between 75 to 100 for more sophisticated vehicles.

• One of the first and very popular use of embedded system in automotive industry was microprocessor based fuel injection.

Some of the other uses of embedded controllers in a vehicle are listed below:

a. Air Conditioner

b. Engine Control

c. Fan Control

d. Headlamp Control

e. Automatic break system control

f. Wiper control

g. Air bag control

h. Power Windows

AES are normally built around microcontrollers or DSPs or a hybrid of the two and are generally known as Electronic Control Units (ECUs).

Types Of Electronic Control Units(ECU)

1. High-speed Electronic Control Units (HECUs):

a. HECUs are deployed in critical control units requiring fast response.

b. They Include fuel injection systems, antilock brake systems, engine control, electronic throttle, steering controls, transmission control and central control units.

2. Low Speed Electronic Control Units (LECUs):-

a. They are deployed in applications where response time is not so critical.

b. They are built around low cost microprocessors and microcontrollers and digital signal processors.

c. Audio controller, passenger and driver door locks, door glass control etc.

Automotive Communication Buses:

Embedded system used inside an automobile communicate with each other using serial buses. This reduces the wiring required. Following are the different types of serial Interfaces used in automotive embedded applications:

a. Controller Area Network (CAN):-

- CAN bus was originally proposed by Robert Bosch.
- It supports medium speed and high speed data transfer

• CAN is an event driven protocol interface with support for error handling in data transmission.

- Generally employed in Air bag control system, poer strain systems like Antilock breaking system and engine control and navigation sytsems like GPS.

b. Local Interconnect Network (LIN):-

• LIN bus is single master multiple slave communication interface with support for data rates up to 20 Kbps and is used for sensor/actuator interfacing.

• LIN bus follows the master communication triggering to eliminate the bus arbitration problem

• LIN bus applications are mirror controls , fan controls , seat positioning controls

c. Media-Oriented System Transport(MOST):-

• MOST is targeted for automotive audio/video equipment interfacing

• A MOST bus is a multimedia fiber optics point–to- point network implemented in a star , ring or daisy chained topology over optical fiber cables

.

• MOST bus specifications define the physical as well as application layer , network layer and media access control.

Fundamental issues in Hardware Software Co – Design

The fundamental issues in solving the problem of hardware software co-design are

(i) Selecting the model
(ii) Selecting the Architecture
(iii) Selecting the language
(iv) Partitioning System Requirements into hardware and software

Selecting the model:

In hardware software co-design, models are used for capturing and describing the system characteristics. A model is a formal system consisting of objects and composition rules. It is hard to make a decision on which model should be followed in a particular system design. Data Flow Graph (DFG) model, State Machine model, Concurrent Process model, Sequential Program model, Object Oriented model, etc. are the commonly used computational models in embedded system design.

Selecting the Architecture:

A model only captures the system characteristics and does not provide information on 'how the system can be manufactured?''. The architecture specifies how a system is going to implement in terms of the number and types of different components and the interconnection among them. Controller architecture, Datapath Architecture, Complex Instmction Set Computing (CISC), Reduced Instmction Set Computing (RISC), Very Long Instmction Word Computing (VLIW), Single Instmction Multiple Data (SIMD), Multiple Instmction Multiple Data (MIMD), etc. are the commonly used architectures in system design.

The controller architecture implements the finite state machine(FSM) model using a state register and two combinational. The state register holds the present state and the combinational circuits implement the logic for next state and output.

The datapath architecture is best suited for implementing the data flow graph(DFG) model where the output is generated as a result of a set of predefined computations on the input data. A datapath represents a channel between the input and output and in datapath architecture the datapath may contain registers, counters, register files, memories and ports along with high speed arithmetic units.

The Finite State Machine Datapath (FSMD) architecture combines the controller architecture with datapath architecture. It implements a controller with datapath. The controller generates the control input whereas the datapath processes the data.

The Complex Instruction Set Computing (CISC) architecture uses an instruction set representing complex operations.

The Very Long Instruction Word (VLIW) architecture implements multiple functional units (ALUs, multipliers, etc.) in the datapath.

Parallel processing architecture implements multiple concurrent Processing Elements (PEs) and each processing element may associate a datapath containing register and local memory. Single Instruction Multiple Data (SIMD) and Multiple Instruction Multiple Data (MIMD) architectures are examples for parallel processing architecture.

Selecting the language:

A programming language captures a 'Computational Model' and maps it into architecture. There is no hard and fast rule to specify this language should be used for capturing this model. A model can be captured using multiple programming languages like C, C++, C#, Java, etc. for software implementations and languages like VHDL, System C, Verilog, etc. for hardware implementations. The only pre-requisite in

selecting a programming language for capturing a model is that the language should capture the model easily.

Partitioning System Requirements into hardware and software:

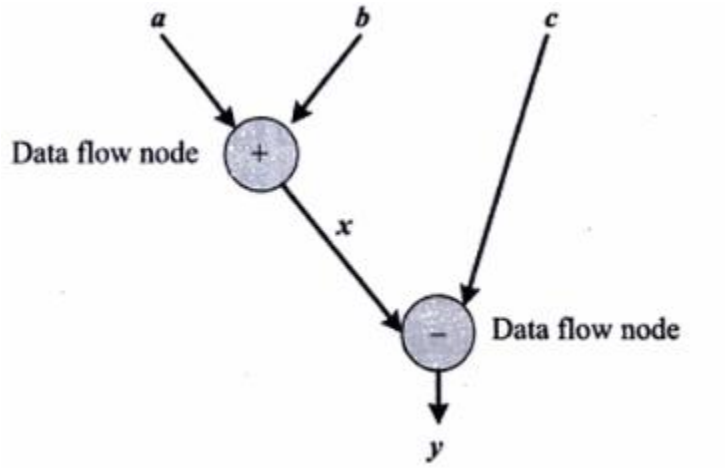
From an implementation perspective, it is possible to implement the system requirements in either hardware or software (firmware). It is a tough decision making task to figure out which one to opt. Various hardware software trade-offs are used for making a decision on the hardware-software partitioning.

Computational Models in Embedded System

Data Flow Graph/Diagram (DFG) Model

The Data Flow Graph (DFG) model translates the data processing requirements into a data flow graph. The Data Flow Graph (DFG) model is a data driven model in which the program execution is determined by data. This model emphasises on the data and operations on the data which transforms the input data to output data. Data Flow Graph (DFG) is a visual model in which the operation on the data (process) is represented using a block (circle) and data flow is represented using arrows. An inward arrow to the process (circle) represents input data and an outward arrow from the process (circle) represents output data in DFG notation.

Embedded applications which are computational intensive and data driven are modeled using the DFG model. DSP applications are typical examples for it. For example, if the computational requirement is x = a + b; and y = x - c. Figure illustrates the DFG model for implementing these requirements

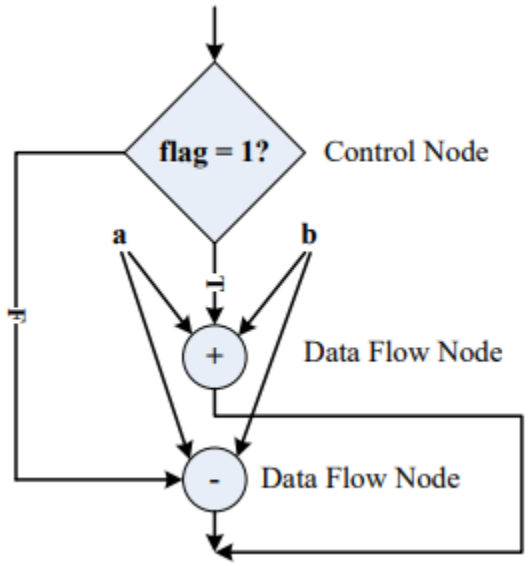


DFG Model for x = a + b; and y = x - c.

Control Data Flow Graph/Diagram (CDFG)

The Control DFG (CDFG) model is used for modelling applications involving conditional program execution. CDFG models contains both data operations and control operations. The CDFG uses Data Flow Graph (DFG) as element and conditional (constructs) as decision makers. CDFG contains both data flow nodes and decision nodes, whereas DFG contains only data flow nodes.

The CDFG model for the requirement **If flag = 1, x = a + b, else y = a-b;** is shown in figure.



CDFG model for **If flag = 1, x = a + b, else y = a-b;**

The control node is represented by a 'Diamond' block which is the decision making element in a normal flow chart based design.

State Machine Model

The State Machine model is used for modelling reactive or event-driven embedded systems whose processing behaviour are dependent on state transitions. The State Machine model describes the system behaviour with 'States', 'Events', 'Actions' and 'Transitions'. State is a representation of a current situation. An event is an input to the state. The event acts as stimuli for state transition. Transition is the movement from one state to another. Action is an activity to be performed by the state machine. A Finite State Machine (FSM) model is one in which the number of states are finite.

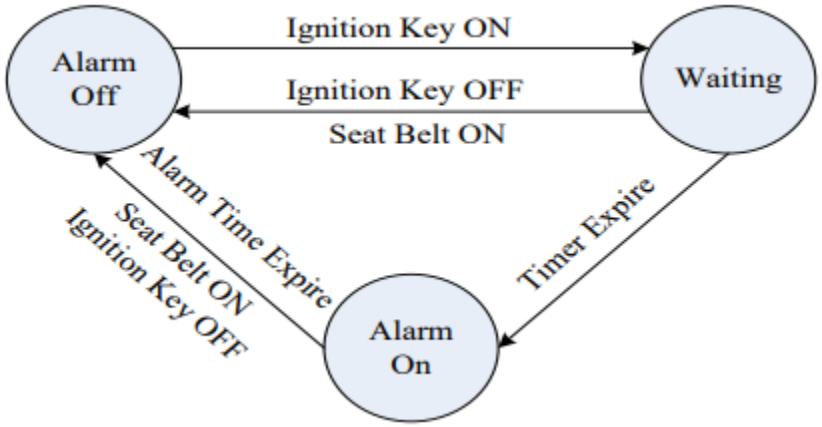
Example: Design of an embedded system for driver/passenger 'Seat Belt Warning' system in an automotive using the FSM model.

The system requirements are captured as.

1. When the vehicle ignition is turned on and the seat belt is not fastened within 10 seconds of ignition ON, the system generates an alarm signal for 5 seconds.

2. The Alarm is turned off when the alarm time (5 seconds) expires or if the driver/passenger fastens the belt or if the ignition switch is turned off, whichever happens first.

The **states used are 'Alarm Off', 'Waiting' and 'Alarm On'** and the **events are 'Ignition Key ON', 'Ignition Key OFF', 'Timer Expire', 'Alarm Time Expire' and 'Seat Belt ON'.** Using the FSM, the system requirements can be modeled as given in Fig.



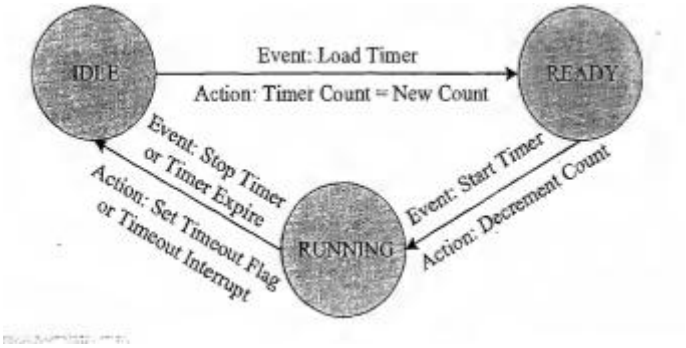
The 'Ignition Key ON' event triggers the 10 second timer and transitions the state to 'Waiting'.

If a 'Seat Belt ON' or 'Ignition Key OFF' event occurs during the wait state, the state transitions into 'Alarm Off'.

When the wait timer expires in the waiting state, the event 'Timer Expire' is generated and it transitions the state to 'Alarm On' from the 'Waiting' state.

The 'Alarm On' state continues until a 'Seat Belt ON' or 'Ignition Key OFF' event or 'Alarm Time Expire' event, whichever occurs first. The occurrence of any of these events transits the state to 'Alarm Off'.

The wait state is implemented using a timer. The timer also has certain set of states and events for state transitions. Using the FSM model, the timer can be modeled as shown in Fig.



The timer state can be either 'IDLE' or 'READY' or 'RUNNING'.

It is said to be in the 'IDLE' state during the normal condition (i.e.) when the timer is not running.

The timer is said to be in the 'READY' state when the timer is loaded with the count corresponding to the required time delay. The timer remains in the 'READY' state until a 'Start Timer' event occurs.

The timer changes its state to 'RUNNING' from the 'READY' state on receiving a 'Start Timer' event and remains in the 'RUNNING' state until the timer count expires or a 'Stop Timer' even occurs.

The timer state changes to 'IDLE' from 'RUNNING' on receiving a 'Stop Timer' or 'Timer Expire' event.

Sequential Program Model

The functions or processing requirements are executed in sequence same as the conventional procedural programming. .

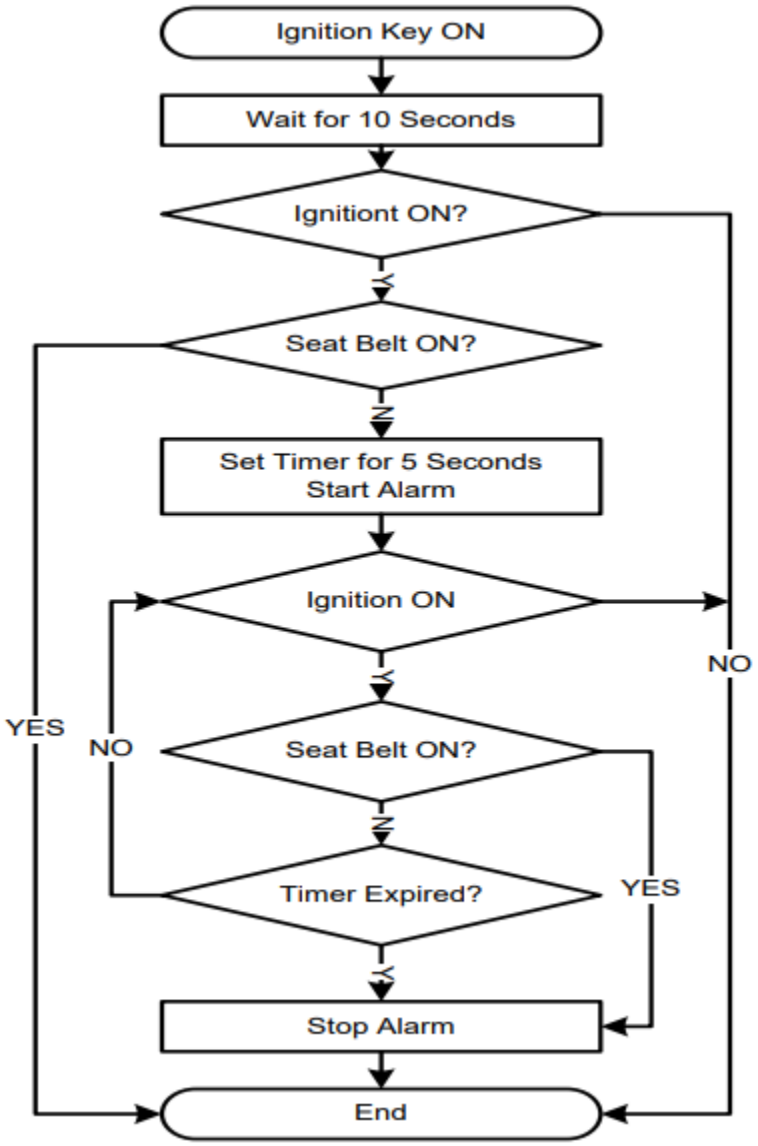
The program instructions are iterated and executed conditionally and the data gets transformed through a series of operations.

FSMs are good choice for sequential program modelling. Flow charts are another important tool used for modelling sequential programs.

The FSM approach represents the states, events, transitions and actions, whereas the Flow Chart models the execution flow. The execution of functions in a sequential program model for the 'Seat Belt Warning' system is illustrated below.

```
# define ON 1

# define OFF 0

# define YES 1

# define NO 0

Void seat_belt_warn()

{ wait_10Sec ();

If(check_ignitionkey()==ON)

{ If(check_seat_belt()==OFF)

{ start_timer(5);

Start_alarm();

While((check_seat_belt()==OFF) && (check_ignitionkey()==OFF) && (timer_expire()==
NO));

Stop_alarm();

}}}
```

Sequential Programming model for Seat belt warning System



Flowchart for Seat belt warning System

Concurrent/Communicating Process Model

The concurrent or communicating process model models concurrently executing tasks/processes.

It is easier to implement certain requirements in concurrent processing model than the conventional sequential execution.

Sequential execution leads to a single sequential execution of task and thereby leads to poor processor utilisation, when the task involves I/O waiting, sleeping for specified duration etc.

In Concurrent processing the task is split into multiple subtasks and it is possible to tackle the CPU usage effectively, when the subtask under execution goes to a wait or sleep mode, by switching the task execution.

Concurrent processing model requires additional overheads in task scheduling, task synchronisation and communication.

Example: 'Seat Belt Warning' system in concurrent processing model.

The processing requirements are splitted into multiple subtasks:

1. Timer task for waiting 10 seconds (wait timer task)

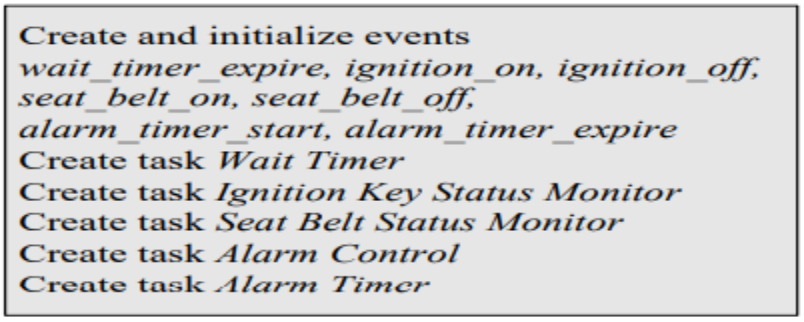
2. Task for checking the ignition key status (ignition key status monitoring task)

3. Task for checking the seat belt status (seat belt status monitoring task)

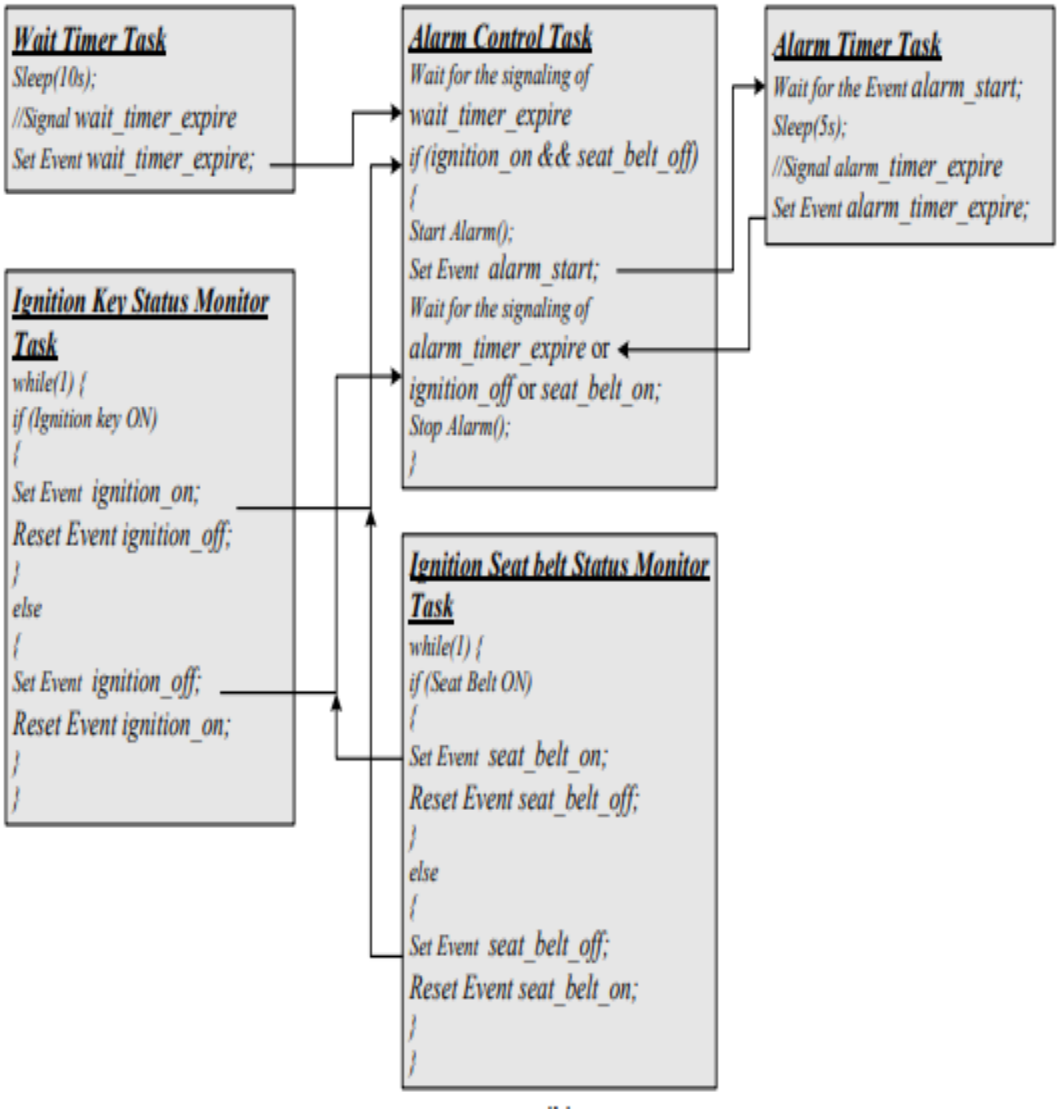
4. Task for starting and stopping the alarm (alarm control task)

5. Alarm timer task for waiting 5 seconds (alarm timer task)

- Tasks are executed concurrently
- 'Events' are used for synchronizing the execution of tasks



(a)



Embedded Firmware Design & Development

- The embedded firmware is responsible for controlling the various peripherals of the embedded hardware and generating response in accordance with the functional requirements of the product.
- The embedded firmware is usually stored in a permanent memory (ROM) and it is non-alterable by end users.
- Designing embedded firmware requires understanding of the particular embedded product hardware, like various component interfacing, memory map details, I/O port

details, configuration and register details of various hardware chips used and some programming language (either low level Assembly Language or High level language like C/C++ or a combination of the two).

- The embedded firmware development process starts with the conversion of the firmware requirements into a program model using various modelling tools.
- There exist two basic approaches for the design and implementation of embedded firmware, namely
  - The **Super loop** based approach
  - The **Embedded Operating System** based approach

Embedded firmware Design Approaches:

The Super loop based Approach:

- Suitable for applications that are not time critical and where the response time is not so important (Embedded systems where missing deadlines are acceptable)
- Very similar to a conventional procedural programming where the code is executed task by task
- The tasks are executed in a never ending loop. The task listed on top on the program code is executed first and the tasks just below the top are executed after completing the first task
- A typical super loop implementation will look like:

1. *Configure the common parameters and perform initialization for various hardware components memory, registers etc.*
2. *Start the first task and execute it*
3. *Execute the second task*
4. *Execute the next task*
5. *……*
6. *……..*
7. *Execute the last defined task*
8. *Jump back to the first task and follow the same flow*

The operational sequence listed above can be visualised in C Program code as

```
Void main()
{
Configurations();
Initialisations();
While(1)
{
Task1();
Task 2();
…….
…….
Task n();
}
}
```

Advantages:

- The 'Super loop based design' doesn't require an operating system, since there is no need for task scheduling
- Simple and straight forward without any OS related overheads.
- Can be deployed in low-cost embedded products and products where response time is not time critical.
- Requires less memory footprint

Drawbacks

- Any failure in any part of a single task will affect the total system.
- The lack of real timeliness (As the number of tasks are increasing, the frequency in which the task gets the CPU also increases)

Embedded firmware Design Approaches – Embedded OS based Approach

- The embedded device contains an Embedded Operating System which can be one of:
  - A Real Time Operating System (RTOS)

- A Customized General Purpose Operating System (GPOS)

- The Embedded OS is responsible for scheduling the execution of user tasks and the allocation of system resources among multiple tasks
- Involves lot of OS related overheads apart from managing and executing user defined tasks
- Microsoft® Windows XP Embedded is an example of GPOS for embedded devices
- Point of Sale (PoS) terminals, Gaming Stations, Tablet PCs etc are examples of embedded devices running on embedded GPOSs
- '*Windows CE*', '*Windows Mobile*','*QNX*', '*VxWorks*', '*ThreadX*', '*MicroC/OS-II*', '*Embedded Linux*', '*Symbian*' etc are examples of RTOSs employed in Embedded Product development
- Mobile Phones, PDAs, Flight Control Systems etc are examples of embedded devices that runs on RTOSs

Embedded firmware Development using Assembly Language

- '***Assembly Language***' is the human readable notation of '***machine language***'. '***machine language***' is a processor understandable language.
- Machine language is a binary representation and it consists of 1s and 0s
- Assembly language and machine languages are processor/controller dependent
- An Assembly language program written for one processor/controller family will not work with others
- Assembly language programming is the process of writing processor specific machine code in mnemonic form, converting the mnemonics into actual processor instructions (machine language) and associated data using an assembler
- Each line of an assembly language program is split into four fields as:

LABEL OPCODE OPERAND COMMENTS

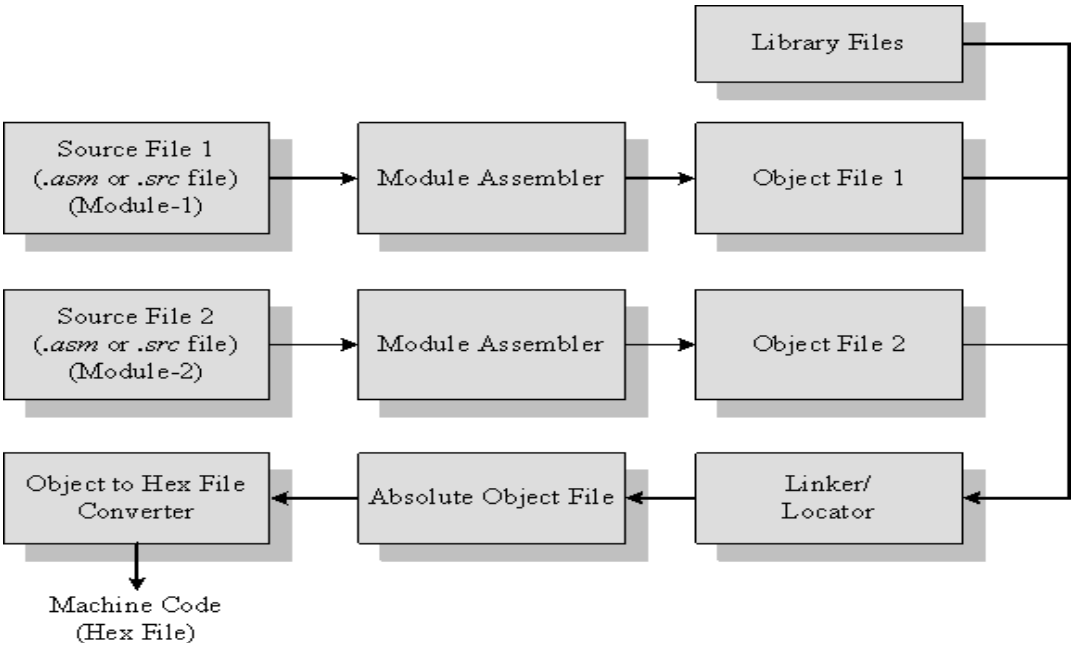
- LABEL is an optional field. A 'LABEL' is an identifier used extensively in programs to reduce the reliance on programmers for remembering where data or code is located. LABEL is commonly used for representing
- A memory location, address of a program, sub-routine, code portion etc.
- The maximum length of a label differs between assemblers. Assemblers insist strict formats for labeling. Labels are always suffixed by a colon and begin with a valid

character. Labels can contain number from 0 to 9 and special character _ (underscore).

```
DELAY:  MOV   R0, #255      ; Load Register R0 with 255
        DJNZ  R1, DELAY     ; Decrement R1 and loop till R1= 0
        RET                 ; Return to calling program
```

Assembly Source File to Hex File Translation

- The Assembly language program written in assembly code is saved as .asm (Assembly file) file or a .src (source) file or a format supported by the assembler
- Similar to 'C' and other high level language programming, it is possible to have multiple source files called modules in assembly language programming. Each module is represented by a '.asm' or '.src' file or the assembler supported file format similar to the '.c' files in C programming
- The software utility called 'Assembler' performs the translation of assembly code to machine code
- The assemblers for different family of target machines are different. A51 Macro Assembler from Keil software is a popular assembler for the 8051 family micro controller
- Each source file can be assembled separately to examine the syntax errors and incorrect assembly instructions
- Assembling of each source file generates a corresponding object file. The object file does not contain the absolute address of where the generated code needs to be placed (a re-locatable code) on the program memory
- The software program called linker/locater is responsible for assigning absolute address to object files during the linking process
- The Absolute object file created from the object files corresponding to different source code modules contain information about the address where each instruction needs to be placed in code memory
- A software utility called 'Object to Hex file converter' translates the absolute object file to corresponding hex file (binary file)



Advantages of Assembly Language Based Development:

Assembly Language based development was (is©) the most common technique adopted from the beginning of embedded technology development.

1. **Efficient Code Memory and Data Memory Usage:**

Since the developer is well versed with the target processor architecture and memory organisation, optimised code can be written for performing operations. This leads to less utilisation of code memory and efficient utilisation of data memory by improving the system performance.

2. **Low Level Hardware Access:**

Most of the code for low level programming like accessing external device specific registers from the operating system kernel, device drivers, and low level interrupt routines, etc. are making use of direct assembly coding since low level device specific operation support is not commonly available with most of the high-level language cross compilers.

3. **Code Reverse Engineering Reverse engineering:**

It is the process of understanding the technology behind a product
by extracting the information from a finished product.

Drawbacks of Assembly Language Based Development:

1. **High Development Time:**

   Assembly language is much harder to program than high level languages. The developer must pay attention to more details and must have thorough knowledge of the architecture, memory organisation and register details of the target processor in use. Learning the inner details of the processor and its assembly instructions is highly time consuming and it creates a delay impact in product development.

2. **Developer Dependency**:

   There is no common written rule for developing assembly language based applications whereas all high level languages instruct certain set of rules for application development. Well documenting the assembly code is a solution for reducing the developer dependency in assembly language programming. If the code is too large and complex, documenting all lines of code may not be productive.

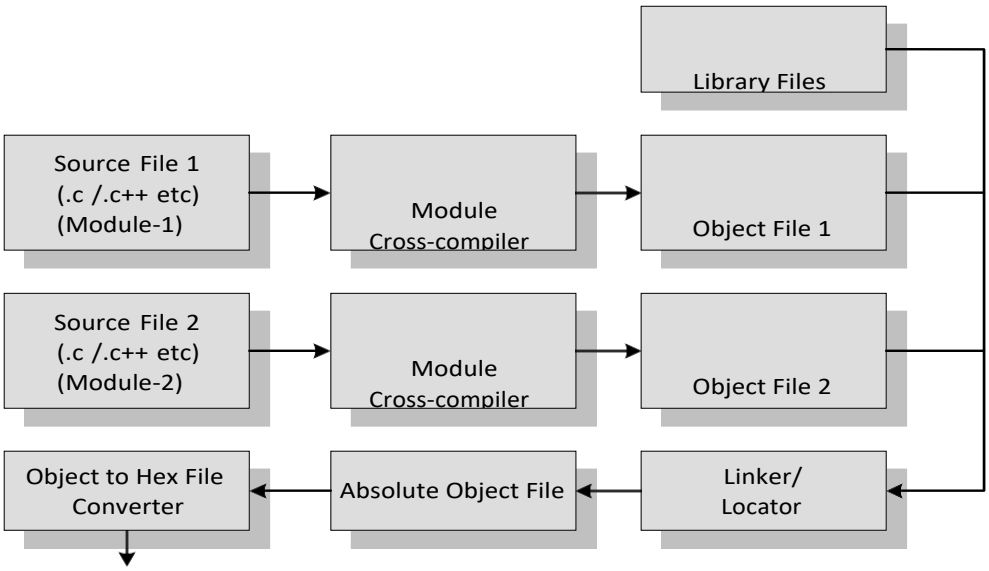
3. **Non-Portable:**

   Target applications written in assembly instructions are valid only for that particular family of processors (e.g. Application written for Intel x86 family of processors) and cannot be re-used for another target processors/controllers (Say ARM 11 family of processors).

Embedded firmware Development using High-level Language

- The embedded firmware is written in any high level language like C, C++
- A software utility called 'cross-compiler' converts the high level language to target processor specific machine code.
- The cross-compilation of each module generates a corresponding object file. The object file does not contain the absolute address of where the generated code needs to be placed (a re-locatable code) on the program memory.
- The software program called linker/locater is responsible for assigning absolute address to object files during the linking process.
- The Absolute object file created from the object files corresponding to different source code modules contain information about the address where each instruction needs to be placed in code memory

- A software utility called 'Object to Hex file converter' translates the absolute object file to corresponding hex file (binary file)



Advantages:

- Reduced Development time
- Developer independency
- Portability

Embedded firmware Development- Mixing of Assembly Language with High Level Language

- Certain situations in embedded firmware development may require the mixing of Assembly Language with high level language or vice versa. Interrupt handling, source code is already available in high level language\Assembly Language etc are examples.
- High Level language and low level language can be mixed in three different ways

- Mixing Assembly Language with High level language like 'C'
- Mixing High level language like 'C' with Assembly Language
- In line Assembly

- The passing of parameters and return values between the high level and low level language is cross-compiler specific.

'C' V/s Embedded C

- '*C*' is a well-structured, well defined and standardized general purpose programming language with extensive bit manipulation support.
- '*C*' offers a combination of the features of high level language and assembly and helps in hardware access programming (system level programming) as well as business package developments (Application developments like pay roll systems, banking applications etc).
- The conventional '*C*' language follows ANSI standard and it incorporates various library files for different operating systems.
- A platform (Operating System) specific application, known as, compiler is used for the conversion of programs written in '*C*' to the target processor (on which the OS is running) specific binary files.
- Embedded C can be considered as a subset of conventional 'C' language
- Embedded C supports all 'C' instructions and incorporates a few target processor specific functions/instructions.
- The standard ANSI 'C' library implementation is always tailored to the target processor/controller library files in Embedded C.
- The implementation of target processor/controller specific functions/instructions depends upon the processor/controller as well as the supported cross-compiler for the particular Embedded C language.
- A software program called 'Cross-compiler' is used for the conversion of programs written in Embedded C to target processor/controller specific instructions.

Compiler V/s Cross-Compiler

- Compiler is a software tool that converts a source code written in a high level language on top of a particular operating system running on a specific target processor architecture (E.g. Intel x86/Pentium).
- The operating system, the compiler program and the application making use of the

source code run on the same target processor.

- The source code is converted to the target processor specific machine instructions
- A native compiler generates machine code for the same machine (processor) on which it is running.
- Cross compiler is the software tools used in cross-platform development applications.
- In cross-platform development, the compiler running on a particular target processor/OS converts the source code to machine code for a target processor whose architecture and instruction set is different from the processor on which the compiler is running or for an operating system which is different from the current development environment OS.
- Embedded system development is a typical example for cross-platform development where embedded firmware is developed on a machine with Intel/AMD or any other target processors and the same is converted into machine code for any other target processor architecture (E.g. 8051, PIC, ARM9 etc).

  Keil C51compiler from Keil software is an example for cross-compiler for 8051 family architecture.